

Line Breaks, Whitespace, & Notepad++ Specials

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\R</code>	Universal newline — matches any line break sequence (<code>\r\n</code> on Windows, <code>\n</code> on Unix, <code>\r</code> on Mac) EXAMPLE: Replace <code>'\R'</code> with <code>' '</code> to join all lines into a single line
<code>\r\n</code>	Windows CRLF line ending explicitly (Carriage Return <code>\r</code> followed by Line Feed <code>\n</code>) EXAMPLE: <code>^.*\r\n</code> matches entire line including CRLF
<code>\n</code> and <code>\r</code>	Line feed (<code>\n</code> , ASCII 10, Unix) and Carriage return (<code>\r</code> , ASCII 13, classic Mac) EXAMPLE: <code>\n</code> matches newline character
<code>\t</code>	Tab character (ASCII 9 / horizontal tab) EXAMPLE: <code>\t+</code> matches consecutive tab stops
<code>\s</code> and <code>\S</code>	Whitespace (space, tab, newline) and Non-whitespace character EXAMPLE: <code>^\s+\$</code> matches lines consisting only of spaces/tabs
<code>\h</code> and <code>\H</code>	Horizontal whitespace (spaces and tabs only, excludes newlines) and non-horizontal whitespace EXAMPLE: <code>[^\S\r\n]+</code> or <code>\h+</code> matches horizontal indentation
<code>\v</code> and <code>\V</code>	Vertical whitespace (newline, line feed, vertical tab, form feed) and non-vertical whitespace EXAMPLE: <code>\v+</code> matches consecutive vertical blank lines
<code>\x{HHHH}</code> or <code>\xHH</code>	Hexadecimal character code (e.g. <code>\x{20AC}</code> for €, <code>\x20</code> for space, <code>\x00</code> for null byte) EXAMPLE: <code>\x{00A0}</code> matches non-breaking space

Anchors, Boundaries, & Inline Flags

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>^</code> and <code>\$</code>	Start of line (^) and end of line (\$); matches line boundaries by default in Notepad++ EXAMPLE: <code>^INFO</code> matches lines starting with 'INFO'
<code>\A</code> and <code>\Z</code>	Start of entire file (\A) and absolute end of entire file (\Z) regardless of line count EXAMPLE: <code>\A-- START OF FILE\R</code>
<code>\b</code> and <code>\B</code>	Word boundary (\b) and non-word boundary (\B) at the edge of \w characters EXAMPLE: <code>\bword\b</code> matches exact word 'word'
<code>\K</code>	Keep Out — resets match position and keeps everything matched to the left out of Find/Replace EXAMPLE: Find: <code>'item_id=\K\d+'</code> Replace: <code>'9999'</code>
<code>(?i)</code> and <code>(?-i)</code>	Inline case-insensitive (?i) and case-sensitive (?-i) switch within the pattern EXAMPLE: <code>(?i)error</code> matches ERROR, Error, error
<code>(?s)</code> and <code>(?-s)</code>	Inline dot-all mode — (?s) enables dot (.) matching newlines; (?-s) disables it (overrides UI checkbox) EXAMPLE: <code>(?s)<div>.*?</div></code> matches multi-line HTML div
<code>(?m)</code> and <code>(?-m)</code>	Inline multiline flag — controls whether ^ and \$ match individual line boundaries EXAMPLE: <code>(?m)^\d+</code>
<code>(?x)</code>	Extended mode — ignores unescaped spaces and permits # comments inside the search pattern EXAMPLE: <code>(?x)\d{4} # Year \R</code>

Replace Field Tokens & Case Modifiers

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>\$1, \$2, \$3 ... (or \1, \2)</code>	Inserts the text matched by the 1st, 2nd, 3rd parenthesized capture group in Find pattern EXAMPLE: Find: <code>'(\w+) (\w+)'</code> Replace: <code>'\$2, \$1'</code>
<code>\$0 (or & or \0)</code>	Inserts the entire matched text into the replacement string EXAMPLE: Find: <code>'\d+'</code> Replace: <code>'[\$0]'</code>
<code>\U ... \E</code>	Converts all enclosed replacement text to UPPERCASE until \E terminator EXAMPLE: Find: <code>'(\w+)'</code> Replace: <code>'\U\$1\E' → 'hello' becomes 'HELLO'</code>
<code>\L ... \E</code>	Converts all enclosed replacement text to lowercase until \E terminator EXAMPLE: Find: <code>'([A-Z]+)'</code> Replace: <code>'\L\$1\E' → 'HELLO' becomes 'hello'</code>
<code>\u and \l</code>	Converts only the single next character to uppercase (\u) or lowercase (\l) EXAMPLE: Find: <code>'(\w+)'</code> Replace: <code>'\u\L\$1\E' (Capitalizes / Title Cases word)</code>
<code>\${name} (or \k<name>)</code>	Inserts the text captured by a named capture group (?<name>...) EXAMPLE: Find: <code>'(?<id>\d+)'</code> Replace: <code>'ID: \${id}'</code>
<code>(?{1}then:else) / (?1then:else)</code>	Conditional replacement — if group 1 matched, outputs 'then', otherwise outputs 'else' EXAMPLE: Find: <code>'(\w+) (\d+)'</code> Replace: <code>'(?{1}Word:Number)'</code>
<code>\r\n or \n (in Replace)</code>	Inserts an actual Windows CRLF or Unix LF line break in the replacement field EXAMPLE: Find: <code>','</code> Replace: <code>'\r\n'</code> (converts comma list to lines)

Groups, Lookarounds, & Atomic Matching

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>(pattern)</code>	Numbered capturing group — captures matched subexpression for backreferencing and \$1 replace EXAMPLE: <code>(\d{4})-(\d{2})-(\d{2})</code>
<code>(?:pattern)</code>	Non-capturing group — groups subexpressions for quantifiers without saving \$1 capture memory EXAMPLE: <code>(?:https? ftp)://\S+</code>
<code>(?<name>pattern)</code>	Named capture group — assigns custom name to group for \${name} replacement EXAMPLE: <code>(?<year>\d{4})-(?<month>\d{2})</code>
<code>(?=pattern)</code> and <code>(?!pattern)</code>	Positive lookahead (?=...) and Negative lookahead (?!...) assertions (zero-width match) EXAMPLE: <code>\d+(?=px)</code> matches number only before 'px'
<code>(?<=pattern)</code> and <code>(?<!pattern)</code>	Positive lookbehind (?<=...) and Negative lookbehind (?<!...) assertions EXAMPLE: <code>(?<= \\$)\d+</code> matches number only after '\$'
<code>(>pattern)</code> and possessive quantifiers <code>++</code> , <code>*+</code>	Atomic grouping / possessive quantifiers — locks in matches without backtracking for faster search EXAMPLE: <code>(>\w+):</code> or <code>\w++:</code>
<code>(?:branch1 branch2)</code>	Branch reset group — alternatives share the exact same capture group numbering (\$1) EXAMPLE: <code>(?:'([^\']*)' "([^\"]*)"")</code>